AI VIET NAM
@aivietnam.edu.vn

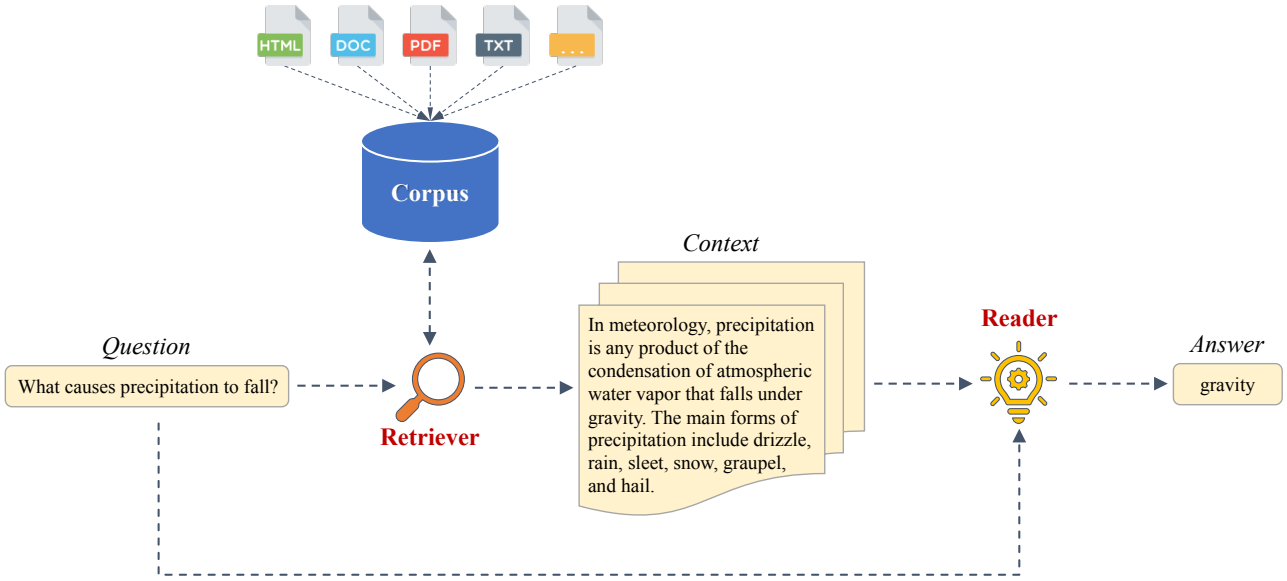
AI VIET NAM – AI COURSE 2025

Tutorial: End-to-end Question Answering

Dương Đình Thắng Nguyễn Anh Khôi Vũ Yến Linh Đinh Quang Vinh

I. Giới thiệu

End-to-end Question Answering là một bài toán thuộc lĩnh vực Xử lý ngôn ngữ tự nhiên liên quan đến việc xây dựng một hệ thống hỏi đáp sử dụng công cụ truy vấn để tìm kiếm các tài liệu có liên quan đến câu hỏi, từ đó lựa chọn câu trả lời phù hợp nhất đến người dùng. End-to-end Question Answering cũng là một ý tưởng tiền đề khởi nguồn cho một trong những khái niệm phổ biến trong khía cạnh cải thiện các mô hình ngôn ngữ lớn (LLMs), còn được gọi là Retrieval Augmented Generation (RAG).

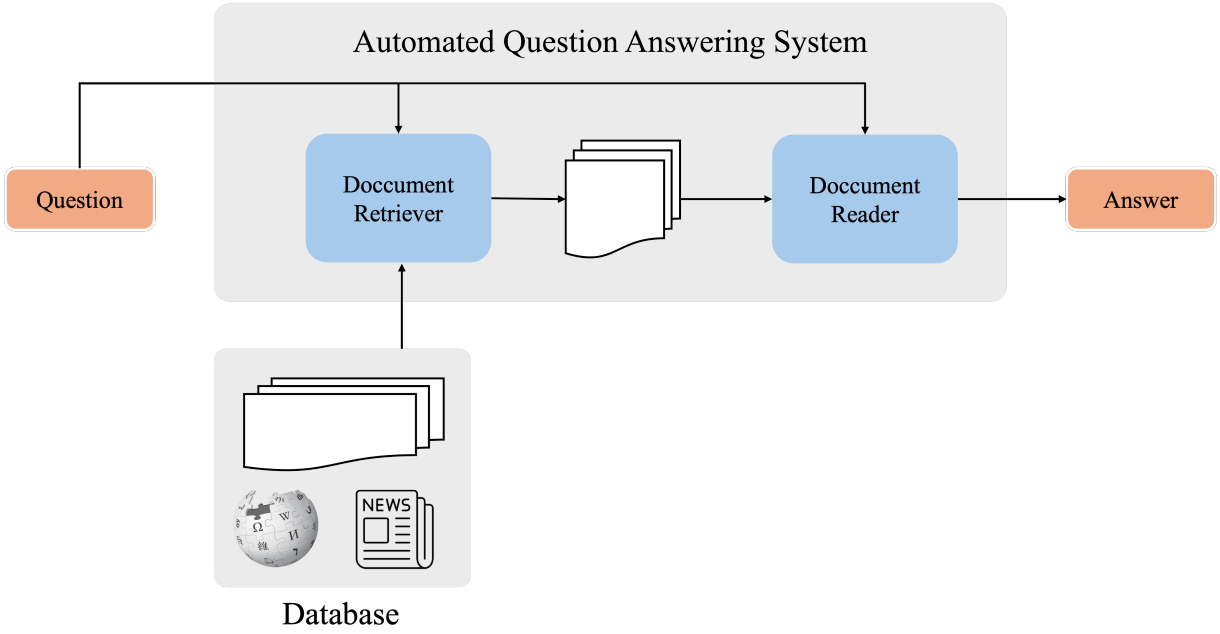


Hình 1: Ảnh minh hoạt tổng quát về một hệ thống End-to-end QA.

Trong project này, chúng ta sẽ cùng tìm hiểu việc phát triển một hệ thống end-to-end hỏi đáp tự động, với khả năng trả lời một câu hỏi với nội dung bất kì. Theo đó, Input và output của chương trình như sau:

- **Input:** Một câu hỏi.
- **Output:** Câu trả lời tương ứng.

Cụ thể, ta sẽ xây dựng chương trình dựa vào dataset SQuAD2.0, một bộ dữ liệu về đọc hiểu, vector database là FAISS và mô hình BERT để thực hiện các nhiệm vụ cụ thể trong chương trình. Để xây dựng một chương trình End-to-end Question Answering, chúng ta cần hoàn thiện hai module chính bao gồm Retriever và Reader.



Hình 2: Minh họa pipeline của bài toán End-to-end Question Answering.

- **Retriever:** Thành phần truy vấn các tài liệu có liên quan đế câu hỏi có trong cơ sở dữ liệu.
- **Reader:** Dựa trên các tài liệu truy vấn được và câu hỏi đầu vào, trích xuất vùng văn bản chứa câu trả lời chính xác nhất và gửi tới người dùng.

II. Cài đặt chương trình

Trong phần này, quá trình cài đặt toàn bộ chương trình End-to-end Question Answering được mô tả như sau:

II.1. Dataset SQuAD2.0

Stanford Question Answering Dataset (SQuAD) là bộ data theo hướng đọc hiểu, bao gồm các đoạn văn (passage) khác nhau về nhiều chủ đề, ứng với mỗi đoạn văn sẽ có một các câu hỏi ngắn kèm theo. Bảng 1 miêu tả cấu trúc chi tiết về dataset SQuAD2.0:

Context: The English name "Normans" comes from the French words Normans/Normanz, plural of Normant, modern French normand, which is itself borrowed from (Old Low Franconian Nortmann "Northman" or directly from Old Norse Nordmaðr, Latinized variously as Nortmannus, Normannus, or Nordmannus (recorded in Medieval Latin, **9th century**) to mean "Norseman, **Viking**".

Q1: What causes precipitation to fall?
A1: Viking

Q2: When was the Latin version of the word Norman first recorded?
A2: 9th century

Q3: When was the French version of the word Norman first recorded?
A3: No Answers

Context: In meteorology, precipitation is any product of the condensation of atmospheric water vapor that falls under **gravity**. The main forms of precipitation include drizzle, rain, sleet, snow, **graupel** and hail... Precipitation forms as smaller droplets coalesce via collision with other rain drops or ice crystals **within a cloud**. Short, intense periods of rain in scattered locations are called “showers”.

Q1: What causes precipitation to fall?
A1: Gravity

Q2: What is another main form of precipitation be- sides drizzle, rain, snow, sleet and hail?
A2: Graupel

Q3: Where do water droplets collide with ice crystals to form precipitation?
A3: Within a cloud

Hình 3: Ví dụ minh họa về dataset SQuAD2.0.

Câu trả lời cho các câu hỏi ngắn là những từ/cụm từ có sẵn trong đoạn văn cho trước (không yêu cầu suy luận phức tạp), hoặc các câu hỏi không trả lời được dựa vào đoạn văn (answer là no answer). Bảng 2 thống kê về dataset SQuAD2.0:

II.2. Reader: DistilBERT

Đầu tiên ta sẽ xây dựng model Reader hay chính là model QA trong project này.

II.2.1. Install and import bibraries

Đầu tiên ta sẽ install một số thư viện cần thiết mà Colab chưa hỗ trợ.

```
!pip install -qq datasets==2.16.1 evaluate==0.4.1 transformers[sentencepiece]==4.35.2
!pip install -qq accelerate==0.26.1
!apt install git-lfs
```

	SQuAD 2.0
Train	
Total examples	130,319
Negative examples	43,498
Total articles	442
Articles with negatives	285
Development	
Total examples	11,873
Negative examples	5,945
Total articles	35
Articles with negatives	35
Test	
Total examples	8,862
Negative examples	4,332
Total articles	28
Articles with negatives	28

Bảng 1: Thống kê số lượng sample của dataset SQuAD2.0.

Answer type	Percentage	Example
Date	8.9%	19 October 2023
Other Numeric	10.9%	12
Person	12.9%	Thomas Coke
Location	4.4%	Germany
Other Entity	15.3%	ABC Sports
Common Noun Phrase	31.8%	property damage
Adjective Phrase	3.9%	second-largest
Verb Phrase	5.5%	returned to Earth
Clause	3.7%	to avoid trivialization
Other	2.7%	quietly

Bảng 2: Thống kê các loại câu trả lời khác nhau của dataset SQuAD2.0.

Sau đó ta sẽ tiến hành login vào HuggingFace để download dataset và model có sẵn. Khi chạy block code này thì HuggingFace sẽ đưa ra một đường dẫn đến trang HuggingFace để lấy mã token.

```
from huggingface_hub import notebook_login

notebook_login()
```

Cuối cùng ta sẽ import các thư viện chính được sử dụng trong phần này:

```
import numpy as np
from tqdm.auto import tqdm
import collections

import torch

```

```python
from datasets import load_dataset
from transformers import AutoTokenizer
from transformers import AutoModelForQuestionAnswering
from transformers import TrainingArguments
from transformers import Trainer
import evaluate

device = torch.device("cuda") if torch.cuda.is_available() else \
        torch.device("cpu")
```

II.2.2. Setup config

Tiếp theo ta sẽ setup một số config cơ bản:

```python
# Sử dụng mô hình "distilbert-base-uncased" làm mô hình checkpoint
MODEL_NAME = "distilbert-base-uncased"

# Độ dài tối đa cho mỗi đoạn văn bản sau khi được xử lý
MAX_LENGTH = 384

# Khoảng cách giữa các điểm bắt đầu của các đoạn văn bản liên tiếp
STRIDE = 128
```

II.2.3. Setup Dataset

- **Download dataset:**

```python
# Download squad dataset từ HuggingFace
DATASET_NAME = "squad_v2"
raw_datasets = load_dataset(DATASET_NAME)
```

- **Load tokenizer and run some examples:**

```python
# Load tokenizer để run một số example
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
```

II.2.4. Tokenize Dataset

Trong phần này ta sẽ tiến hành tokenize dataset cho tập train và tập val.

- **Tokenize train set:** Hàm `preprocess_training_examples` nhận dữ liệu đào tạo làm đầu vào và tiền xử lý để chuẩn bị cho việc huấn luyện mô hình hỏi đáp. Trong quá trình này, hàm trích xuất danh sách câu hỏi, mã hóa thông tin đầu vào bằng tokenizer, và trích xuất `offset_mapping` và `sample_map` để ánh xạ vị trí từ mã hóa về văn bản gốc. Hàm cũng xác định vị trí bắt đầu và kết thúc của câu trả lời trong ngữ cảnh và thêm thông tin về vị trí này vào biến inputs.

```python
# Định nghĩa hàm preprocess_training_examples và nhận tham số examples
# là dữ liệu training
def preprocess_training_examples(examples):
```

```python
    # Trích xuất danh sách câu hỏi từ examples và
    # loại bỏ các khoảng trắng dư thừa
    questions = [q.strip() for q in examples["question"]]

    # Tiến hành mã hóa thông tin đầu vào sử dụng tokenizer
    inputs = tokenizer(
        questions,
        examples["context"],
        max_length=MAX_LENGTH,
        truncation="only_second",
        stride=STRIDE,
        return_overflowing_tokens=True,
        return_offsets_mapping=True,
        padding="max_length",
    )

    # Trích xuất offset_mapping từ inputs và loại bỏ nó ra khỏi inputs
    offset_mapping = inputs.pop("offset_mapping")

    # Trích xuất sample_map từ inputs và loại bỏ nó ra khỏi inputs
    sample_map = inputs.pop("overflow_to_sample_mapping")

    # Trích xuất thông tin về câu trả lời (answers) từ examples
    answers = examples["answers"]

    # Khởi tạo danh sách các vị trí bắt đầu và kết thúc câu trả lời
    start_positions = []
    end_positions = []

    # Duyệt qua danh sách offset_mapping
    for i, offset in enumerate(offset_mapping):
        # Xác định index của mẫu (sample) liên quan đến offset hiện tại
        sample_idx = sample_map[i]

        # Trích xuất sequence_ids từ inputs
        sequence_ids = inputs.sequence_ids(i)

        # Xác định vị trí bắt đầu và kết thúc của ngữ cảnh
        idx = 0
        while sequence_ids[idx] != 1:
            idx += 1
        context_start = idx
        while sequence_ids[idx] == 1:
            idx += 1
        context_end = idx - 1

        # Trích xuất thông tin về câu trả lời cho mẫu này
        answer = answers[sample_idx]

        if len(answer["text"]) == 0:
            start_positions.append(0)
            end_positions.append(0)
        else:
            # Xác định vị trí ký tự bắt đầu và kết thúc của câu trả lời
            # trong ngữ cảnh
```

```
            start_char = answer["answer_start"][0]
            end_char = answer["answer_start"][0] + len(answer["text"][0])

            # Nếu câu trả lời không nằm hoàn toàn trong ngữ cảnh,
            # gán nhãn là (0, 0)
            if offset[context_start][0] > start_char
               or offset[context_end][1] < end_char:
                start_positions.append(0)
                end_positions.append(0)
            else:
                # Nếu không, gán vị trí bắt đầu và kết thúc dựa trên
                # vị trí của các mã thông tin
                idx = context_start
                while idx <= context_end and offset[idx][0] <= start_char:
                    idx += 1
                start_positions.append(idx - 1)

                idx = context_end
                while idx >= context_start and offset[idx][1] >= end_char:
                    idx -= 1
                end_positions.append(idx + 1)

    # Thêm thông tin vị trí bắt đầu và kết thúc vào inputs
    inputs["start_positions"] = start_positions
    inputs["end_positions"] = end_positions

    return inputs
```

Sau đó ta sẽ chạy đoạn hàm trên với từng dòng trong `raw_dataset` của tập train:

```
# Tạo một biến train_dataset và gán cho nó giá trị sau khi áp dụng
# hàm preprocess_training_examples lên tập dữ liệu "train"
#
# Bật chế độ xử lý theo từng batch bằng cách đặt batched=True
#
# Loại bỏ các cột không cần thiết trong
# tập dữ liệu "train" bằng cách sử dụng remove_columns

train_dataset = raw_datasets["train"].map(
    preprocess_training_examples,
    batched=True,
    remove_columns=raw_datasets["train"].column_names,
)

# In ra độ dài của tập dữ liệu "train" ban đầu và
# độ dài của tập dữ liệu đã được xử lý (train_dataset)
len(raw_datasets["train"]), len(train_dataset)
```

- **Tokenize val set:** Ta sẽ làm tương tự với tập val, hàm `preprocess_validation_examples`- thực hiện việc tiền xử lý dữ liệu cho quá trình đánh giá mô hình. Hàm chuẩn bị danh sách câu hỏi, mã hóa các câu hỏi và văn bản liên quan bằng cách sử dụng tokenizer. Sau đó xác định ví dụ tham chiếu cho từng dòng đầu vào và điều chỉnh ánh xạ offset để loại bỏ các offset không phù hợp với `sequence_ids`. Cuối cùng là thêm thông tin về ví dụ tham chiếu vào đầu vào.

```python
def preprocess_validation_examples(examples):
    # Chuẩn bị danh sách câu hỏi bằng cách
    # loại bỏ khoảng trắng ở đầu và cuối mỗi câu hỏi
    questions = [q.strip() for q in examples["question"]]

    # Sử dụng tokenizer để mã hóa các câu hỏi và văn bản liên quan
    inputs = tokenizer(
        questions,
        examples["context"],
        max_length=MAX_LENGTH,
        truncation="only_second",
        stride=STRIDE,
        return_overflowing_tokens=True,
        return_offsets_mapping=True,
        padding="max_length",
    )

    # Lấy ánh xạ để ánh xạ lại ví dụ tham chiếu cho từng dòng trong inputs
    sample_map = inputs.pop("overflow_to_sample_mapping")
    example_ids = []

    # Xác định ví dụ tham chiếu cho mỗi dòng đầu vào và
    # điều chỉnh ánh xạ offset
    for i in range(len(inputs["input_ids"])):
        sample_idx = sample_map[i]
        example_ids.append(examples["id"][sample_idx])

        sequence_ids = inputs.sequence_ids(i)
        offset = inputs["offset_mapping"][i]

        # Loại bỏ các offset không phù hợp với sequence_ids
        inputs["offset_mapping"][i] = [
            o if sequence_ids[k] == 1 else None \
                for k, o in enumerate(offset)
        ]

    # Thêm thông tin ví dụ tham chiếu vào đầu vào
    inputs["example_id"] = example_ids

    return inputs
```

Ta sẽ chạy đoạn hàm trên với từng dòng trong `raw_dataset` của tập validation:

```python
# Tạo một biến validation_dataset và gán giá trị bằng việc sử dụng dữ liệu
# từ raw_datasets["validation"] sau khi áp dụng một hàm xử lý trước.

validation_dataset = raw_datasets["validation"].map(
    preprocess_validation_examples,
    batched=True,
    remove_columns=raw_datasets["validation"].column_names,
)

# In ra độ dài của raw_datasets["validation"]
# và validation_dataset để so sánh.
len(raw_datasets["validation"]), len(validation_dataset)
```

II.2.5. Train model

Sau khi đã chuẩn bị xong dataset, ta sẽ tiến hành load model từ HuggingFace để chuẩn bị cho quá trình training:

```
# Load model
model = AutoModelForQuestionAnswering.from_pretrained(MODEL_NAME)
```

Tiếp theo ta sẽ định nghĩa một số parameter mà ta sẽ sử dụng để training model:

```
# Tạo đối tượng args là các tham số cho quá trình huấn luyện
args = TrainingArguments(
    output_dir="distilbert-finetuned-squadv2", # Thư mục lưu output
    evaluation_strategy="no",  # Chế độ đánh giá không tự động sau mỗi epoch
    save_strategy="epoch",  # Lưu checkpoint sau mỗi epoch
    learning_rate=2e-5,  # Tốc độ học
    num_train_epochs=3,  # Số epoch huấn luyện
    weight_decay=0.01,  # Giảm trọng lượng mô hình để tránh overfitting
    fp16=True,  # Sử dụng kiểu dữ liệu half-precision để tối ưu tài nguyên
    push_to_hub=True,  # Đẩy kết quả huấn luyện lên HuggingFace Hub
)
```

Cuối cùng ta sẽ khởi tạo class Trainer, đây là class chính để training model, ta sẽ không cần phải định nghĩa hàm train, đưa input vào mode, tính toán loss, update gradient nữa, hàm class này sẽ tự động làm giúp chúng ta. Sau khi đã khởi tạo thì chỉ cần gọi `trainner.train()` thì quá trình training model sẽ được tiến hành:

```
# Khởi tạo một đối tượng Trainer để huấn luyện mô hình
trainer = Trainer(
    model=model,  # Sử dụng mô hình đã tạo trước đó
    args=args,  # Các tham số và cấu hình huấn luyện
    train_dataset=train_dataset,  # Sử dụng tập dữ liệu huấn luyện
    eval_dataset=validation_dataset,  # Sử dụng tập dữ liệu đánh giá
    tokenizer=tokenizer,  # Sử dụng tokenizer để xử lý văn bản
)

# Bắt đầu quá trình huấn luyện
trainer.train()
```

[49354/49410 1:39:37 < 00:06, 8.26 it/s, Epoch 3.00/3]

Step	Training Loss
500	3.102900
1000	2.268800
1500	1.971300
2000	1.810200
2500	1.702800
3000	1.625000
3500	1.574800
4000	1.566400
4500	1.495000
5000	1.480800
5500	1.438300
6000	1.402200
6500	1.418000
7000	1.426900
45500	0.686000
46000	0.696700
46500	0.674500
47000	0.651100
47500	0.676700
48000	0.651300
48500	0.673300
49000	0.697000

Hình 4: Kết quả training loss tại mỗi 500 step.

Sau khi quá trình training hoàn tất, ta sẽ đưa weight, config của model lên HuggingFace Hub để lưu lại:

```
# Gửi dữ liệu đào tạo lên Hub
trainer.push_to_hub(commit_message="Training complete")
```

II.2.6. Evaluate model

Để đánh giá performance của model ta sẽ sử dụng metric squad từ thư viện evaluate:

```
# Tải metric "squad" từ thư viện evaluate
metric = evaluate.load("squad_v2")
```

Hàm `compute_metrics()` nhận các đầu vào như `start_logits`, `end_logits`, features, và examples, và thực hiện các bước sau để tính toán các độ đo và kết quả đánh giá mô hình hỏi đáp. Trong quá trình tính toán, hàm này tạo một danh sách các câu trả lời dự đoán dựa trên các logits được dự đoán bởi mô hình. Điều này bao gồm việc xác định vị trí bắt đầu và kết thúc tốt nhất cho các câu trả lời và đánh giá xem chúng có hợp lệ hay không dựa trên độ dài tối đa cho câu trả lời. Cuối cùng, hàm tính toán các độ đo và trả về kết quả đánh giá mô hình hỏi đáp dựa trên các câu trả lời dự đoán và câu trả lời lý thuyết từ ví dụ.

```
N_BEST = 20 # Số lượng kết quả tốt nhất được lựa chọn sau khi dự đoán
MAX_ANS_LENGTH = 30 # Độ dài tối đa cho câu trả lời dự đoán

def compute_metrics(start_logits, end_logits, features, examples):
    # Tạo một từ điển mặc định để ánh xạ mỗi ví dụ
    # với danh sách các đặc trưng tương ứng
```

```python
    example_to_features = collections.defaultdict(list)
    for idx, feature in enumerate(features):
        example_to_features[feature["example_id"]].append(idx)

    predicted_answers = []
    for example in tqdm(examples):
        example_id = example["id"]
        context = example["context"]
        answers = []

        # Lặp qua tất cả các đặc trưng liên quan đến ví dụ đó
        for feature_index in example_to_features[example_id]:
            start_logit = start_logits[feature_index]
            end_logit = end_logits[feature_index]
            offsets = features[feature_index]["offset_mapping"]

            # Lấy các chỉ số có giá trị lớn nhất cho start và end logits
            start_indexes = np.argsort(start_logit)[-1:-N_BEST-1:-1].tolist()
            end_indexes = np.argsort(end_logit)[-1:-N_BEST-1:-1].tolist()
            for start_index in start_indexes:
                for end_index in end_indexes:
                    # Bỏ qua các câu trả lời
                    # không hoàn toàn nằm trong ngữ cảnh
                    if offsets[start_index] is None or \
                        offsets[end_index] is None:
                        continue
                    # Bỏ qua các câu trả lời có độ dài > max_answer_length
                    if end_index - start_index + 1 > MAX_ANS_LENGTH:
                        continue

                    # Tạo một câu trả lời mới
                    text = context[
                        offsets[start_index][0]:offsets[end_index][1]
                    ]
                    logit_score = start_logit[start_index] + \
                                  end_logit[end_index]
                    answer = {
                        "text": text,
                        "logit_score": logit_score,
                    }
                    answers.append(answer)

        # Chọn câu trả lời có điểm số tốt nhất
        if len(answers) > 0:
            best_answer = max(answers, key=lambda x: x["logit_score"])
            answer_dict = {
                "id": example_id,
                "prediction_text": best_answer["text"],
                "no_answer_probability": 1 - best_answer["logit_score"]
            }
        else:
            answer_dict = {
                "id": example_id,
                "prediction_text": "",
                "no_answer_probability": 1.0
```

```
            }
        predicted_answers.append(answer_dict)

    # Tạo danh sách câu trả lời lý thuyết từ các ví dụ
    theoretical_answers = [
        {"id": ex["id"], "answers": ex["answers"]} for ex in examples
    ]
    # Sử dụng metric.compute để tính toán các độ đo và trả về kết quả
    return metric.compute(
        predictions=predicted_answers,
        references=theoretical_answers
    )
```

Sau khi đã định nghĩa hàm evaluation, ta sẽ tiến hành predict model trên tập validation rồi đưa vào hàm `compute_metrics`:

```
# Thực hiện dự đoán trên tập dữ liệu validation
predictions, _, _ = trainer.predict(validation_dataset)

# Lấy ra thông tin về các điểm bắt đầu và
# điểm kết thúc của câu trả lời dự đoán
start_logits, end_logits = predictions

# Tính toán các chỉ số đánh giá sử dụng hàm compute_metrics
results = compute_metrics(
    start_logits,
    end_logits,
    validation_dataset,
    raw_datasets["validation"]
)
results
```

```
100% ■■■
```

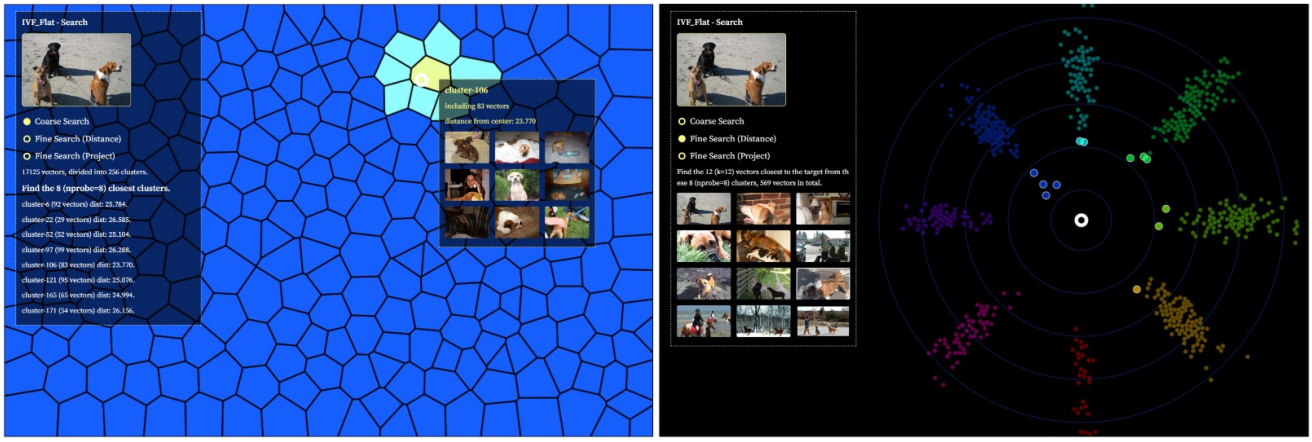
```python
from transformers import pipeline

PIPELINE_NAME = "question-answering"
MODEL_NAME = "thangduong0509/distilbert-finetuned-squadv2"
pipe = pipeline(PIPELINE_NAME, model=MODEL_NAME)
```

Sau đây ta sẽ chạy thử một example để test model:

```python
INPUT_QUESTION = "What is my name?"
INPUT_CONTEXT = "My name is AI Vietnam and I live in Vietnam."
pipe(question=INPUT_QUESTION, context=INPUT_CONTEXT)

## >> Output: {"score": 0.97179114818573, "start": 11, "end": 21, "answer": "AI Vietnam"}
```

II.3. Retriever: Faiss

Faiss (Facebook AI Similarity Search) là một thư viện được phát triển bởi Facebook AI Research Team, hỗ trợ trong việc tìm kiếm tương đồng và phân cụm (clustering) các vector với tốc độ và độ chính xác cao. Các bạn có thể đọc thêm về Faiss tại đây.

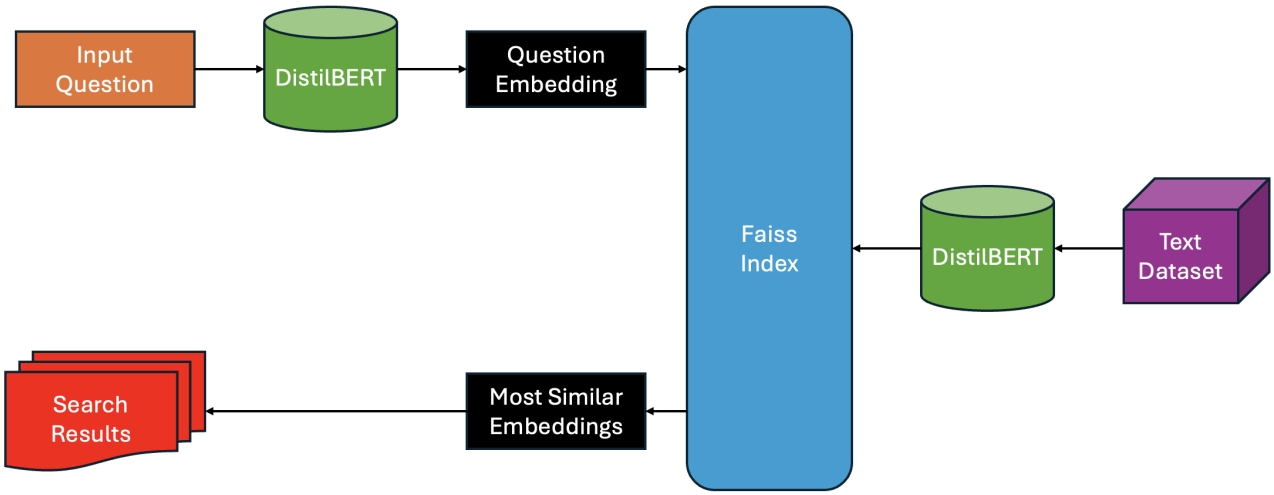


Hình 5: Minh hoạ phương pháp lập chỉ mục (indexing) IVF_Flat trong thư viện Faiss. Source.

Tại đây, chúng ta sẽ ứng dụng Faiss để làm module Retriever cho hệ thống E2E QA của chúng ta. Với nhiệm vụ tìm kiếm các context phù hợp nhất cho câu hỏi đầu vào, ta sẽ cài đặt Faiss theo cách thức như sau:

1. Với bộ dữ liệu SQuAD2.0, ta sẽ xây dựng một database chứa thêm cột đại diện cho vector embedding của câu hỏi.
2. Thực hiện embedding các câu hỏi sử dụng DistilBERT.
3. Thực hiện tìm kiếm tương đồng giữa các vector trong cột mới và vector câu hỏi đầu vào, từ đó tìm ra nội dung context có liên quan nhất.

Quy trình xử lý của Faiss trong bài có thể được tóm gọn qua ảnh sau:



Hình 6: Minh họa các bước xây dựng một vector database với Faiss.

Để cài đặt Faiss phục vụ cho việc tìm kiếm các văn bản context của các câu hỏi có nội dung giống với câu hỏi đầu vào, ta thực hiện như sau:

II.3.1. Cài đặt và import các thư viện cần thiết

```
!pip install -qq transformers[sentencepiece]==4.35.2 datasets==2.16.1 evaluate==0.4.1
!sudo apt-get install libomp-dev
!pip install -qq faiss-gpu
```

```
import numpy as np
import collections
import torch
import faiss
import evaluate

from datasets import load_dataset
from transformers import AutoTokenizer, AutoModel
from transformers import AutoModelForQuestionAnswering
from transformers import TrainingArguments
from transformers import Trainer
from tqdm.auto import tqdm

device = torch.device("cuda") if torch.cuda.is_available() else torch.device("cpu")
```

II.3.2. Tải bộ dữ liệu

Ta tải bộ dữ liệu SQuAD2.0:

```
DATASET_NAME = "squad_v2"
raw_datasets = load_dataset(DATASET_NAME, split="train+validation")
```

```
raw_datasets
```

Để tận dụng toàn bộ ngữ liệu, chúng ta sẽ gom hai bộ dữ liệu train và validation trong bước tạo vector database này.

II.3.3. Loại bỏ các mẫu không có đáp án

Các mẫu dữ liệu không có đáp án thường chứa các câu hỏi không liên quan đến đoạn văn ngữ cảnh. Vì vậy, ta sẽ loại bỏ các trường hợp này ra khỏi bộ dữ liệu:

```
raw_datasets = raw_datasets.filter(
    lambda x: len(x["answers"]["text"]) > 0
)
```

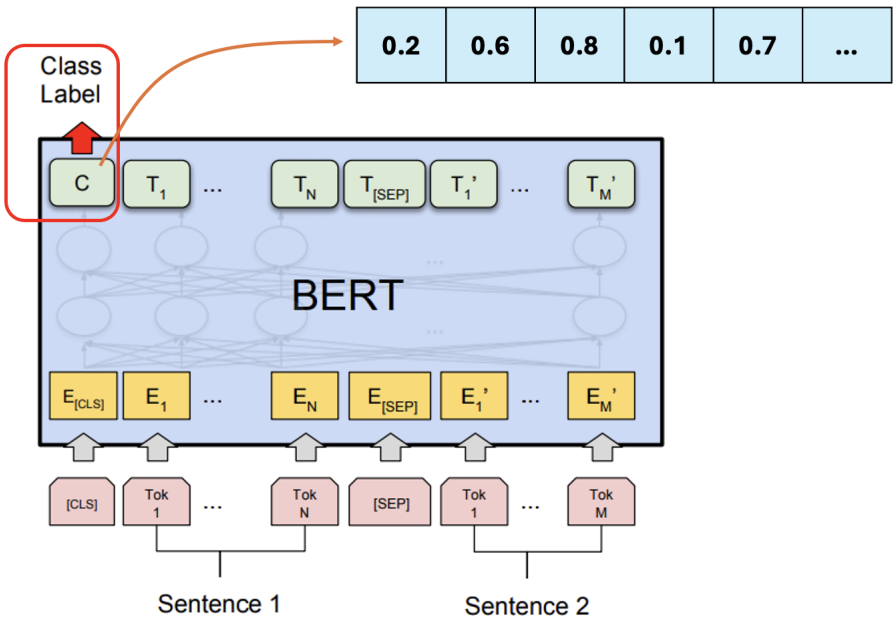
II.3.4. Khởi tạo mô hình

Để tạo vector embedding cho các câu hỏi, ta sẽ sử dụng mô hình pre-trained DistilBERT:

```
MODEL_NAME = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = AutoModel.from_pretrained(MODEL_NAME).to(device)
```

II.3.5. Xây dựng hàm lấy vector embedding:

Ở đây, ta sẽ xây dựng một hàm trả về vector embedding cho một đoạn text đầu vào, cụ thể ở đây sẽ là câu hỏi. Để tạo vector embedding đại diện cho câu hỏi, ta sẽ sử dụng vector hidden state từ token `[CLS]` trong kết quả output của mô hình DistilBERT:



Hình 7: Ảnh minh họa vị trí của final hidden state của token `CLS`.

Đầu tiên, ta xây dựng hàm lấy final hidden state của token `CLS`:

```
def cls_pooling(model_output):
    return model_output.last_hidden_state[:, 0]
```

Sau đó, xây dựng hàm đưa một văn bản vào model để từ đó có thể gọi hàm `cls_pooling()`:

```
def get_embeddings(text_list):
    encoded_input = tokenizer(
        text_list,
        padding=True,
        truncation=True,
        return_tensors="pt"
    )
    encoded_input = {k: v.to(device) for k, v in encoded_input.items()}
    model_output = model(**encoded_input)

    return cls_pooling(model_output)
```

II.3.6. Xây dựng vector database

Với hàm tạo vector embedding đã triển khai, ta sẽ sử dụng nó để tạo một cột trong bảng dữ liệu dùng để chứa kết quả lời gọi hàm `get_embeddings()` với input là các câu hỏi của từng mẫu dữ liệu. Cụ thể, ta tạo cột mới tên là `question_embedding` và lưu vector embedding của câu hỏi như sau:

```
EMBEDDING_COLUMN = "question_embedding"
embeddings_dataset = raw_datasets.map(
    lambda x: {
        EMBEDDING_COLUMN: get_embeddings(
            x["question"]
        ).detach().cpu().numpy()[0]
    }
)
```

Sau đó, để có thể tìm kiếm các vector tương đồng, ta sẽ tạo một cấu trúc dữ liệu đặc biệt là Faiss Index như sau:

```
embeddings_dataset.add_faiss_index(column=EMBEDDING_COLUMN)
```

Cuối cùng, chúng ta đã có một vector database lưu trữ vector embedding của các câu hỏi trong bộ dữ liệu. Từ đây, ta sẽ tiến hành thử thực hiện truy vấn với một câu hỏi đầu vào bất kì như sau:

```
input_question = "When did Beyonce start becoming popular?"

input_quest_embedding = get_embeddings([input_question])
input_quest_embedding = input_quest_embedding.cpu().detach().numpy()

TOP_K = 5
scores, samples = embeddings_dataset.get_nearest_examples(
    EMBEDDING_COLUMN, input_quest_embedding, k=TOP_K
)

```

```
for idx, score in enumerate(scores):
    print(f"Top {idx + 1}\tScore: {score}")
    print(f"Question: {samples["question"][idx]}")
    print(f"Context: {samples["context"][idx]}")
    print()
```

Khi chạy xong đoạn lệnh trên, ta được kết quả trả về như sau:

```
Top 1   Score: 0.0
Question: When did Beyonce start becoming popular?
Context: Beyoncé Giselle Knowles-Carter (/biːˈjɒnseɪ/ bee-YON-say) (born September 4, 1981) is an American singer, songwriter,

Top 2   Score: 2.6135313510894775
Question: When did Beyoncé rise to fame?
Context: Beyoncé Giselle Knowles-Carter (/biːˈjɒnseɪ/ bee-YON-say) (born September 4, 1981) is an American singer, songwriter,

Top 3   Score: 4.859482288360596
Question: When did Beyoncé release Formation?
Context: On February 6, 2016, one day before her performance at the Super Bowl, Beyoncé released a new single exclusively on mu

Top 4   Score: 5.054221153259277
Question: In which decade did Beyonce become famous?
Context: Beyoncé Giselle Knowles-Carter (/biːˈjɒnseɪ/ bee-YON-say) (born September 4, 1981) is an American singer, songwriter,

Top 5   Score: 5.170375347137451
Question: When did Beyonce begin her deals with name brands?
Context: The release of a video-game Starpower: Beyoncé was cancelled after Beyoncé pulled out of a $100 million with GateFive
```

Hình 8: Kết quả truy vấn được in ra màn hình.

Có thể thấy, vì câu hỏi đầu vào có tồn tại trong vector database của chúng ta nên mẫu dữ liệu tương đồng nhất cũng chính là mẫu có câu hỏi tương tự.

II.4. Áp dụng mô hình hỏi-đáp để trả lời câu hỏi

Như vậy, chúng ta đã có hai thành phần Retriever và Reader. Chúng ta sẽ viết một đoạn code ngắn để thực hiện chương trình End-to-End QA. Đầu tiên, khởi tạo mô hình hỏi-đáp đã huấn luyện:

```
from transformers import pipeline

PIPELINE_NAME = "question-answering"
MODEL_NAME = "thangduong0509/distilbert-finetuned-squadv2"
pipe = pipeline(PIPELINE_NAME, model=MODEL_NAME)
```

Tận dụng kết quả truy vấn vừa rồi (nằm ở biến `scores` và `samples`), chúng ta sẽ truyền vào mô hình QA hai thông tin gồm question và context:

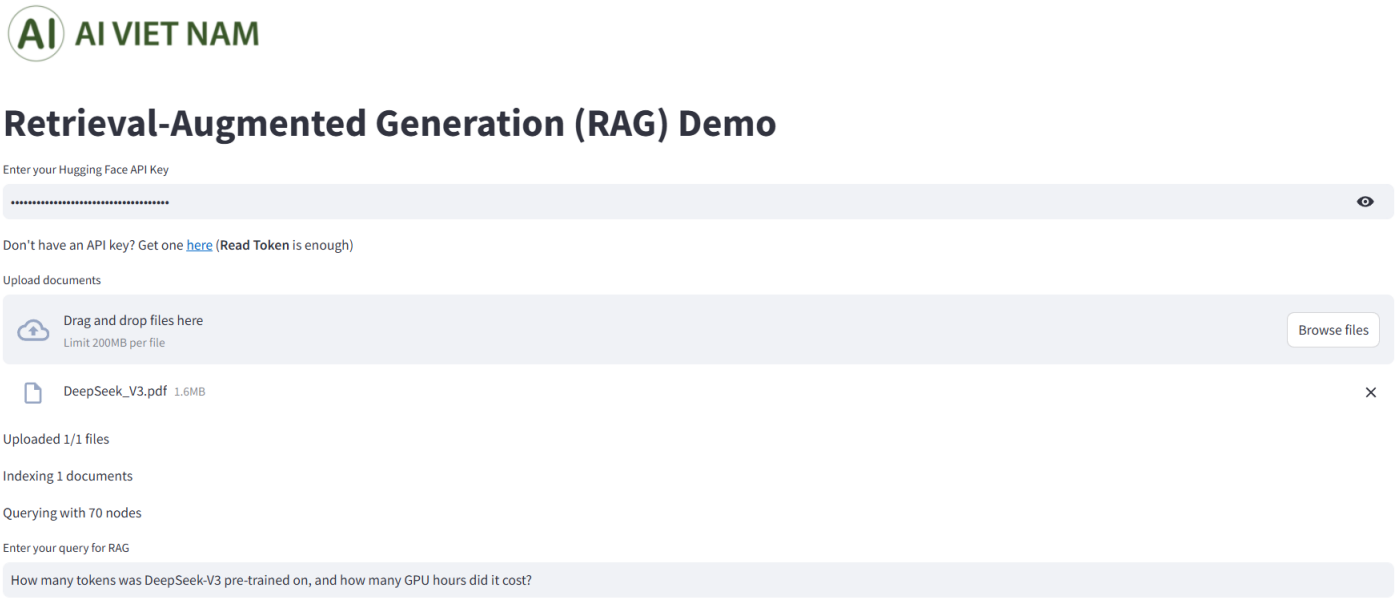
```
print(f"Input question: {input_question}")
for idx, score in enumerate(scores):
    question = samples["question"][idx]
    context = samples["context"][idx]
    answer = pipe(
        question=question,
        context=context
    )
    print(f"Top {idx + 1}\tScore: {score}")
    print(f"Context: {context}")
```

```
    print(f"Answer: {answer}")
    print()
```

```
Input question: When did Beyonce start becoming popular?
Top 1   Score: 0.0
Context: Beyoncé Giselle Knowles-Carter (/biːˈjɒnseɪ/ bee-YON-say) (born September 4, 1981) is an American singer, songwriter,
Answer: {'score': 0.6091989278793335, 'start': 276, 'end': 286, 'answer': 'late 1990s'}

Top 2   Score: 2.6135313510894775
Context: Beyoncé Giselle Knowles-Carter (/biːˈjɒnseɪ/ bee-YON-say) (born September 4, 1981) is an American singer, songwriter,
Answer: {'score': 0.6103343963623047, 'start': 276, 'end': 286, 'answer': 'late 1990s'}

Top 3   Score: 4.859482288360596
Context: On February 6, 2016, one day before her performance at the Super Bowl, Beyoncé released a new single exclusively on mu
Answer: {'score': 0.9715896248817444, 'start': 3, 'end': 19, 'answer': 'February 6, 2016'}

Top 4   Score: 5.054221153259277
Context: Beyoncé Giselle Knowles-Carter (/biːˈjɒnseɪ/ bee-YON-say) (born September 4, 1981) is an American singer, songwriter,
Answer: {'score': 0.7218101620674133, 'start': 281, 'end': 286, 'answer': '1990s'}

Top 5   Score: 5.170375347137451
Context: The release of a video-game Starpower: Beyoncé was cancelled after Beyoncé pulled out of a $100 million with GateFive
Answer: {'score': 0.6330415606498718, 'start': 433, 'end': 452, 'answer': 'since the age of 18'}
```

Hình 9: Kết quả E2E QA được in ra màn hình.

Với mô hình xây dựng được, ta có thể triển khai thành một ứng dụng web demo chương trình cho phép trả lời những câu hỏi xoay quanh một tài liệu PDF mà chúng ta đưa vào. Thông tin về web demo có thể được tìm thấy tại phần IV.



Hình 10: Demo Retrieval-augmented Generation (RAG) (1).

Enter your query for RAG

How many tokens was DeepSeek-V3 pre-trained on, and how many GPU hours did it cost?

Retrieved Documents

Document 1 (Score: 0.8498)

Training Costs Pre-Training Context Extension Post-Training Total in H800 GPU Hours 2664K 119K 5K 2788K in USD $5.328M$0.238M $0.01M$5.576M Table 1 |Training costs of DeepSeek-V3, assuming the rental price of H800 is $2perGPUhour.andgenerationlength.WeevaluateDeepSeek-V3onacomprehensivearrayofbenchmarks.Despiteitseconomicaltrainingcosts,comprehensiveevaluationsrevealthatDeepSeek-V3-Basehasemergedasthestrongestopen-sourcebasemodelcurrentlyavailable,especiallyincodeandmath.Itschatversionalsooutperformsotheropen-sourcemodelsandachievesperformancecomparabletoleadingclosed-sourcemodels,includingGPT-4oandClaude-3.5-Sonnet,onaseriesofstandardandopen-endedbenchmarks.Lastly,weemphasizeagaintheeconomicaltrainingcostsofDeepSeek-V3,summarizedinTable1,achievedthroughouroptimizedco-designofalgorithms,frameworks,andhardware.Duringthepre-trainingstage,trainingDeepSeek-V3oneachtrillintokensrequiresonly180KH800GPUhours,i.e.,3.7daysonourclusterwith2048H800GPUs.Consequently,ourpre-trainingstageiscompletedinlessthantwomonthsandcosts2664KGPUhours.Combinedwith119KGPUhoursforthecontextlengthextensionand5KGPUhoursforpost-training,DeepSeek-V3costsonly2.788MGPUhoursforitsfulltraining.AssumingtherentalpriceoftheH800GPUis2$$ per GPU hour, our total training costs amount to only $5.576M. Note that the aforementioned costs include only the official training of DeepSeek-V3, excluding the costs associated with prior research and ablation experiments on architectures, algorithms, or data. Our main contribution includes: Architecture: Innovative Load Balancing Strategy and Training Objective • On top of the efficient architecture of DeepSeek-V2, we pioneer an auxiliary-loss-free strategy for load balancing, which minimizes the performance degradation that arises from encouraging load balancing.

Document 2 (Score: 0.7705)

Document 3 (Score: 0.7564)

RAG Response:

DeepSeek-V3 was pre-trained on 14.8 trillion tokens, and it cost 2.664M H800 GPU hours. The Multi-Token Prediction (MTP) objective used in DeepSeek-V3 extends the prediction scope to multiple future tokens at each position, which densifies the training signals and may improve data efficiency. The MTP implementation consists of D sequential modules to predict D additional tokens, with each module having a shared embedding layer, a shared output head, a Transformer block, and a projection matrix.

Hình 11: Demo Retrieval-augmented Generation (RAG) (2).

III. Câu hỏi trắc nghiệm

Câu 1. Tại sao mô hình Transformer được sử dụng phổ biến trong bài toán Question Answering (QA)?

A. Do Transformer không có khả năng tự học đặc trưng từ văn bản tự nhiên.

B. Do Transformer chứa nhiều kiến thức về dữ liệu.

C. Có sử dụng mô hình tìm kiếm context.

D. Nắm bắt hiệu quả hơn các phụ thuộc xa (long-range dependencies) thông qua cơ chế self-attention.

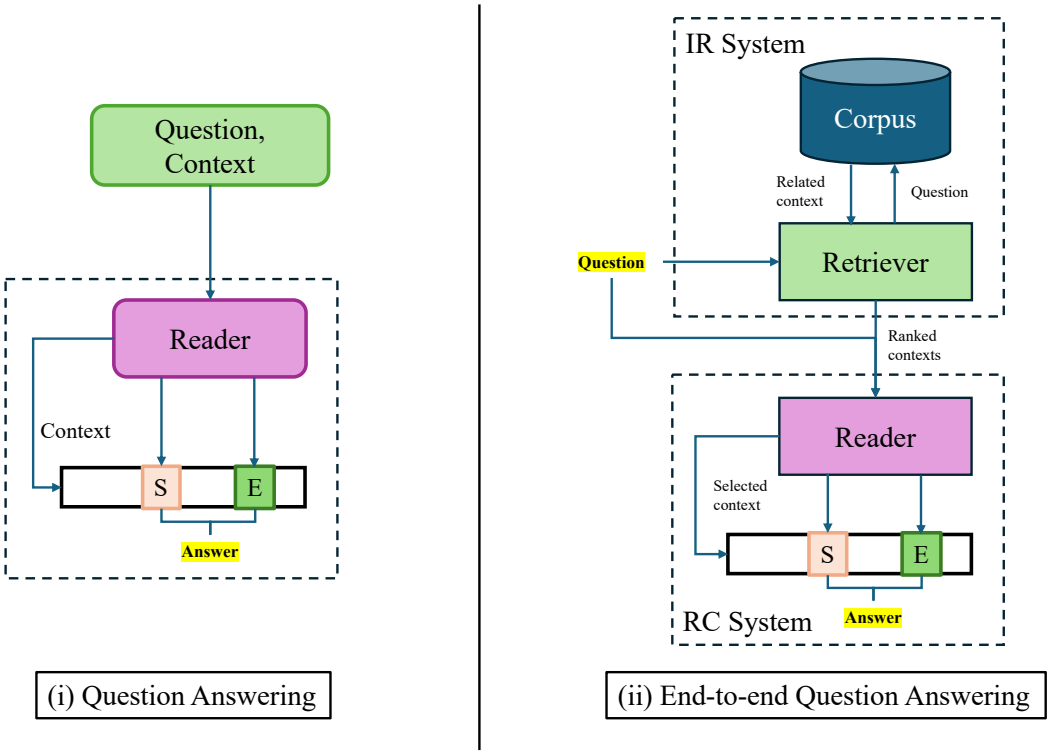
Câu 2. Khi huấn luyện một mô hình QA, tại sao ta thường sử dụng transfer learning?

A. Kế thừa biểu diễn ngôn ngữ đã được học từ các kho dữ liệu khổng lồ.

B. Loại bỏ hoàn toàn sự phụ thuộc vào quá trình tinh chỉnh (fine-tuning).

C. Transfer learning làm gia tăng khả năng xử lý của CPU.

D. Transfer learning giúp mô hình tránh hoàn toàn overfitting.



Hình 12: Ảnh minh họa giữa hệ thống QA và End-to-end QA.

Câu 3. Quan sát Hình 12 và cho biết, so với QA, chương trình End-to-end QA có điểm gì khác biệt?

A. Mô hình trích xuất câu hỏi tốt hơn.

B. Sử dụng kiến trúc transformer.

C. Có sử dụng mô hình tìm kiếm context.

D. Tốc độ xử lý nhanh hơn.

Câu 4. Tham số `stride` có nghĩa là gì trong đoạn code sau:

```
inputs = tokenizer(
    question,  # Danh sách các câu hỏi
    context,  # Nội dung liên quan đến câu hỏi
    max_length=MAX_LENGTH,  # Độ dài tối đa cho đầu ra mã hóa
    truncation="only_second",  # Cắt bớt dữ liệu chỉ cho phần thứ hai (context)
    stride=STRIDE,
    return_overflowing_tokens=True,  # Trả về các tokens vượt quá độ dài tối đa
    return_offsets_mapping=True,  # Trả về bản đồ vị trí của các tokens trong văn bản gốc
    padding="max_length" # Điền vào dữ liệu để có cùng độ dài max_length
)
```

A. Độ dài bước nhảy trong trường hợp dữ liệu dài hơn `max_length`.

B. Độ dài bước nhảy trong trường hợp dữ liệu ngắn hơn `max_length`.

C. Độ dài bước nhảy trong trường hợp dữ liệu bằng `max_length`.

D. Độ dài bước nhảy trong trường hợp bất kỳ.

Câu 5. Tham số `fp16=True` có nghĩa là gì trong đoạn code sau:

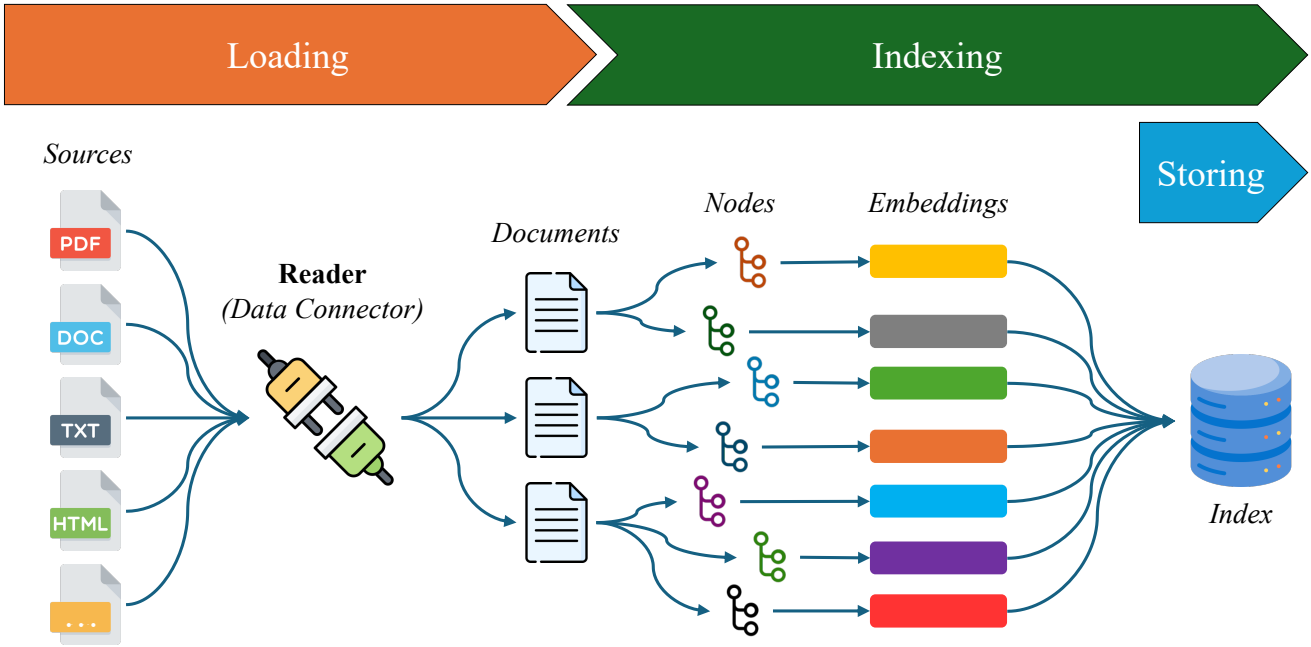
```
# Tạo đối tượng args là các tham số cho quá trình huấn luyện
args = TrainingArguments(
    output_dir="distilbert-finetuned-squadv2",  # Thư mục lưu trữ kết quả huấn luyện
    evaluation_strategy="no",  # Chế độ đánh giá không tự động sau mỗi epoch
    save_strategy="epoch",  # Lưu checkpoint sau mỗi epoch
    learning_rate=2e-5,  # Tốc độ học
    num_train_epochs=3,  # Số epoch huấn luyện
    weight_decay=0.01,  # Giảm trọng lượng mô hình để tránh overfitting
    fp16=True,
    push_to_hub=True,  # Đẩy kết quả huấn luyện lên một nơi chia sẻ trực tuyến (Hub)
)
```

A. Sử dụng kiểu dữ liệu 32-bit để tối ưu hóa tài nguyên.

B. Sử dụng kiểu dữ liệu double để tối ưu hóa tài nguyên.

C. Sử dụng kiểu dữ liệu float để tối ưu hóa tài nguyên.

D. Sử dụng kiểu dữ liệu half-precision để tối ưu hóa tài nguyên.



Hình 13: Ảnh minh họa về việc loading và indexing dữ liệu của một vector database.

Câu 6. Quan sát Hình 13 và cho biết, ưu điểm của vector database khi xử lý các loại dữ liệu phức tạp như hình ảnh, âm thanh và văn bản so với cơ sở dữ liệu quan hệ truyền thống là gì?

A. Nó tự động bóc tách các nguồn tài liệu thành các bảng dữ liệu chuẩn hóa ngặt nghèo để đảm bảo tính toàn vẹn.

B. Nó chuyển đổi dữ liệu thành các biểu diễn *Embeddings*, cho phép truy xuất thông tin dựa trên độ tương đồng về ngữ nghĩa thay vì khớp từ khóa.

C. Nó sử dụng các phân đoạn nhỏ (*Nodes*) để thiết lập hệ thống khóa giao dịch, đảm bảo tính nhất quán tuyệt đối khi xử lý đồng thời.

D. Quá trình lập chỉ mục (*Indexing*) được tối ưu hóa để có thể thực thi trực tiếp các câu lệnh truy vấn SQL phức tạp trên dữ liệu thô.

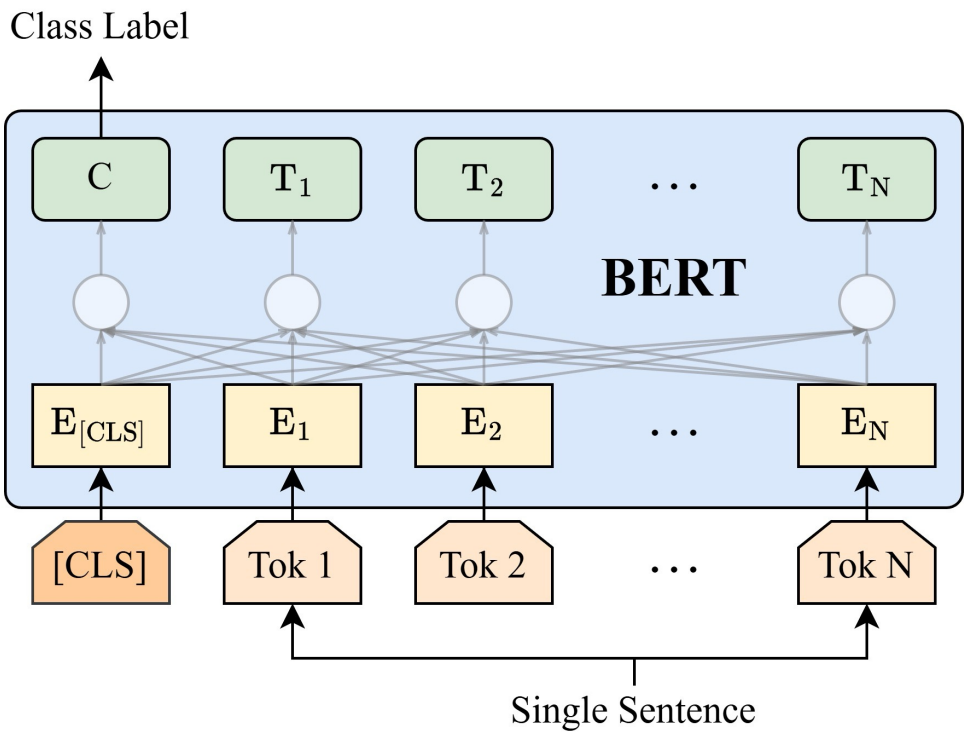
Câu 7. Để tính sự tương đồng giữa hai vector, độ đo nào sau đây không thể áp dụng?

A. Cosine Similarity.

B. Euclidean Distance.

C. Pearson Correlation Coefficient.

D. Maximum Likelihood Estimation.



Hình 14: Ảnh minh họa về kiến trúc mô hình BERT.

Câu 8. Khi ứng dụng BERT (Hình 14) để tạo vector embedding trong vector database, final hidden state của token nào thường được sử dụng trong output của BERT?

A. `[CLS]` token.

B. `[SEP]` token.

C. `[EOS]` token.

D. Final token.

Câu 9. Trong các vector database như Faiss, kỹ thuật nào được sử dụng để nén các vector đa chiều, qua đó giảm thiểu đáng kể dung lượng bộ nhớ (RAM) cần thiết khi lưu trữ ở quy mô lớn?

A. Linear Search.

B. Indexing.

C. Quantization.

D. Encryption.

Câu 10. Trong đoạn code dưới đây:

```
TOP_K = 5
scores, samples = embeddings_dataset.get_nearest_examples(
    EMBEDDING_COLUMN, input_quest_embedding, k=TOP_K
)
```

Biến `TOP_K` còn được hiểu là?

A. Số lượng cluster.

B. Số lượng kết quả trả về.

C. Số epochs.

D. Số chiều trong không gian embedding.

IV. Phụ lục

1. **Hint:** Các file code gợi ý có thể được tải tại đây.
2. **Solution:** Các file code cài đặt hoàn chỉnh và phần trả lời nội dung trắc nghiệm có thể được tải tại đây (Lưu ý: Sáng thứ 3 khi hết deadline phần project, ad mới copy các nội dung bài giải nêu trên vào đường dẫn).
3. **Demo:** Web demo và mã nguồn có thể được truy cập tại đây.
4. **Rubric:**

Mục	Kiến Thức	Đánh Giá
II.	- Kiến thức về bài toán Question Answering. - Kiến thức về bài toán Text Retrieval. - Kiến thức về Open-domain Question Answering. - Kiến thức về cách xây dựng một hệ thống End-to-end Question Answering. - Cách cài đặt, xây dựng và huấn luyện mô hình QA. - Cách cài đặt một vector database.	- Nắm được các khái niệm về bài toán Question Answering, Text Retrieval và End-to-end Question Answering. - Biết cách cài đặt code xây dựng, huấn luyện mô hình QA sử dụng thư viện HuggingFace. - Biết cách cài đặt một vector database để thực hiện tìm kiếm văn bản tương đồng.

- Hết -